

the factorial of 4 (the number $n - 1$). In turn, $4! = 4 \times 3!$, and so on, until you must calculate $1!$, the base case, which is simply 1.

The *factorialByRecursion.py* Python program uses a recursive factorial algorithm:

Python

```
def factorial(number):
    if number == 1:
        # BASE CASE
        return 1
    else:
        # RECURSIVE CASE
        ❶ return number * factorial(number - 1)
print(factorial(5))
```

And the *factorialByRecursion.html* JavaScript program with equivalent code looks like this:

JavaScript

```
<script type="text/javascript">
function factorial(number) {
    if (number == 1) {
        // BASE CASE
        return 1;
    } else {
        // RECURSIVE CASE
        ❶ return number * factorial(number - 1);
    }
}
document.write(factorial(5));
</script>
```

When you run this code to calculate $5!$ recursively, the output matches the iterative program's output:

120

To many programmers, this recursive code looks strange. You know that `factorial(5)` must compute $5 \times 4 \times 3 \times 2 \times 1$, but it's hard to point to the line of code where this multiplication is taking place.